

QLlama: An FPGA-Based Microscaling Quantization Accelerator for Energy-Efficient Llama2 Inference

Hongbing Wen^{ID}, *Student Member, IEEE*, Zihao Wang, Jiale Dong, Wenqi Lou^{ID}, *Member, IEEE*,
Lei Gong^{ID}, Chao Wang^{ID}, *Senior Member, IEEE*, and Xuehai Zhou^{ID}

Abstract—To address the computational power and energy efficiency challenges in Llama2 large-model inference, this letter proposes a hardware-software co-design method and finally implements a high energy efficiency accelerator named QLlama based on FPGA. This work first employs a novel quantization method based on a microscaling data format, which allows sharing a scaling factor with E8M0 format for each subtensor block, thus enabling quantization and dequantization operations to be completed using only shift operations. Second, on this basis, a mixed precision configuration is implemented for different layers of Llama2 to balance accuracy loss and computational efficiency. Finally, a dedicated accelerator QLlama is designed, whose core units include a quantization unit for dynamic quantization, a vector-matrix multiplication unit for high density computation of quantized weights, a scaled dot product unit, and a basic operator unit. Experimental results show that this scheme achieves energy efficiency improvements of 2.13~10.66× with negligible accuracy loss, i.e., <0.2 perplexity increase. The code is available at <https://github.com/wendadawen/QLlama>.

Index Terms—FPGA, inference, Llama2, microscaling (MX) format.

I. INTRODUCTION

WITH the rise of large-language models (LLMs), such as Llama2 [1], their enormous parameter counts have caused inference tasks to face severe computational bottlenecks and memory bandwidth limitations. Traditional GPU platforms can accelerate computations but suffer from high power consumption, making them unsuitable for edge scenarios [2]. FPGAs, with their reconfigurable nature and low-power advantages, have emerged as potential solutions [3]. How to achieve high-energy-efficiency inference through software-hardware co-design while maintaining accuracy has become a critical issue to be urgently addressed.

Current research focuses on two directions: model quantization and hardware acceleration. At the algorithmic level of

quantization, quantization techniques achieve low-bit quantization of weights and activation values, which can significantly reduce storage requirements under the premise of controllable accuracy loss. Methods, such as GPTQ [4] and OWQ [5] have achieved 3–4 bit quantization deployment, but suffer from high hardware complexity. At the hardware level, although existing FPGA accelerators have designed some dedicated computing units for quantized data, the customized hardware adaptation for the data distribution characteristics of the quantized Llama2 large model still shows insufficiency.

We propose QLlama, an FPGA-based quantization inference acceleration scheme for Llama2, which consists of the following components.

- 1) *Microscaling (MX) Data Format Quantization*: A hardware-friendly quantization method is adopted based on MX [6] data format, where subtensor blocks share a scaling factor in the E8M0 format. Quantization and dequantization are implemented via bit shifting, reducing the hardware overhead of multipliers and dividers.
- 2) *Mixed Precision Quantization Algorithm Design*: A mixed-precision MXINT8 quantization quantizes weights while preserving sensitive layers' precision. Activations in weight-activation matrix multiplication are dynamically quantized, with other layers' activations kept at full precision to reduce error accumulation.
- 3) *Hardware Architecture Design and Implementation*: A hierarchical storage architecture is adopted to store quantized weights via HBM pseudo-channel sharding. A dedicated vector-matrix multiplication unit (VMMU) is designed for fast quantized weight computation through block calculation, computational optimization, ping-pong buffering, and memory access optimizations. Custom optimizations are also applied to operators, such as scaled-dot product and dynamic quantization operators.

The experimental results demonstrate that the energy efficiency of QLlama is improved by 10.66×, 2.13×, and 3.38× compared to CPU, GPU, and HLSTransform [7], respectively, with negligible accuracy loss, i.e., <0.2 perplexity increase on the wikitext-2 dataset. This provides an efficient solution for deploying large models in edge scenarios.

II. BACKGROUND

A. Computation Flow

Fig. 1 illustrates the computational flow of QLlama, depicting the decoder block [8] structure repeated multiple times in the

Received 10 August 2025; accepted 14 August 2025. Date of current version 17 October 2025. This work was supported in part by the Strategic Priority Research Program of the Chinese Academy of Sciences under Grant XDB0660101, Grant XDB0660000, and Grant XDB0660100; in part by the National Natural Science Foundation of China under Grant 62172380; in part by the Jiangsu Provincial Natural Science Foundation under Grant BK20241818; in part by the Youth Innovation Promotion Association CAS under Grant Y2021121; and in part by the USTC Research Funds of the Double First-Class Initiative under Grant YD2150002011. This manuscript was recommended for publication by A. Shrivastava. (Corresponding authors: Wenqi Lou; Chao Wang.)

The authors are with the University of Science and Technology of China, Hefei 230026, China, and also with the Suzhou Institute for Advanced Research, University of Science and Technology of China, Suzhou 215123, China (e-mail: louwenqi@ustc.edu.cn; cswang@ustc.edu.cn).

Digital Object Identifier 10.1109/LES.2025.3600563

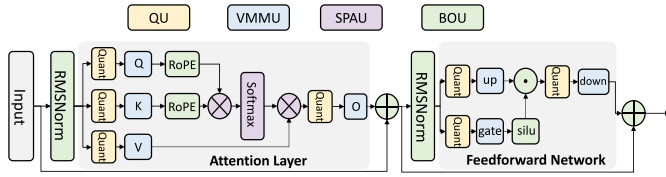


Fig. 1. Computational flow of the Llama2. Different colors correspond to being implemented by different module units.

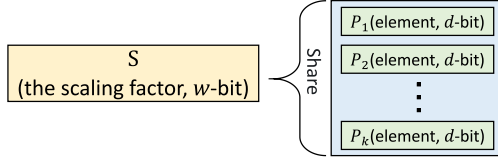


Fig. 2. Single block in a MX data format. The block encodes a vector of k numbers, each with value SP_i .

Llama2 model, which mainly includes two computationally intensive layers: the feedforward network and the attention layer [9]. In the computational flow, two types of vector-matrix multiplications are primarily involved: weight-activation vector-matrix multiplication and activation-activation vector-matrix multiplication. Notably, weight-activation vector-matrix multiplication accounts for most of the computation time in inference tasks. The accelerator performs low bitwidth quantization on both activation values and weights entering the weight-activation vector-matrix multiplication, while preserving the original precision for activation values and weights in other layers. Meanwhile, four modular units are designed to characterize the inference tasks of the Llama2 model, respectively, corresponding to different colors in Fig. 1.

B. Microscaling Data Format

MX data format is a narrow bitwidth data representation method specifically designed for AI computations. Its core advantage lies in achieving a balance between hardware efficiency and precision through a block-level scaling mechanism, thereby reducing storage and computational overheads. As shown in the Fig. 2, this format consists of two parts: a w bit scaling factor S (used to dynamically adjust the numerical range) and a data block. The data block contains k elements (such as P_1 to P_k), all stored in d bits and sharing the same scaling factor S . The data format of the scaling factor is independent of that of the elements, and all k elements adopt the same data format. Each MX block encodes a vector containing k numerical values, where each value is represented as SP_i . Through the joint encoding mechanism of the scaling factor and elements, this format maintains computational precision while saving memory and supports hardware-level efficient computations.

III. ARCHITECTURE AND OPTIMIZATION

A. Overall Architecture

Fig. 3 illustrates the hardware architecture of QLLama, which decomposes the computational flow into four functional units: the VMMU, the scaled dot-product attention unit (SPAU), the quantization unit (QU), and the basic operators unit (BOU). These units are uniformly scheduled by a controller to enable data interaction between functional

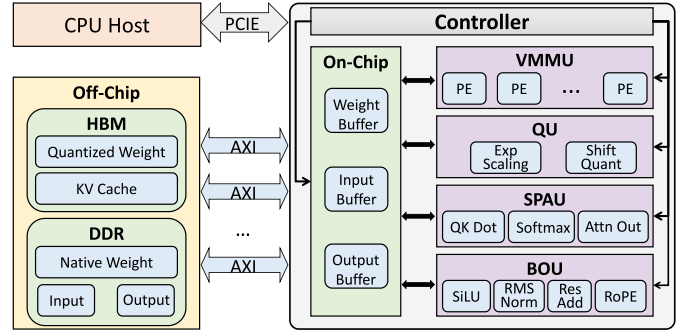


Fig. 3. Overall architecture of accelerator, including host, VMMU, QU, SPAU, and BOU.

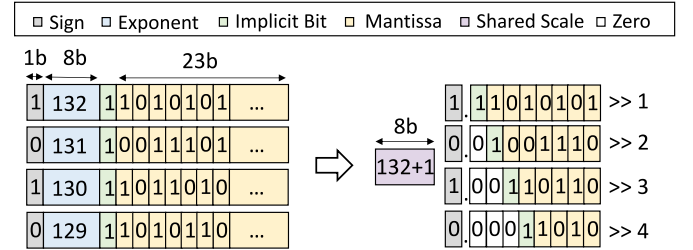


Fig. 4. FP32 data format (left) to MXINT8 data format (right).

units and on-chip/off-chip memories. The VMMU optimized for quantized weights executes activation-weight vector-matrix multiplications. The SPAU performs two activation-activation vector-matrix multiplications followed by row-wise softmax. The QU performs dynamic quantization on the activation tensors input to the VMMU. The BOU integrates model basic components, such as the SiLU [10], RMSNorm [11], residual connections [12], and RoPE [13].

B. Quantization Unit

The QU adopts the MX data format as shown in Fig. 2. Our implementation adopts $k = 32$, $w = 8$, and $d = 8$. The scaling factor S uses the E8M0 data format (8-bit exponent, no mantissa), while the elements P_i use an 8-bit fixed-point format (1 sign bit and 7 fractional bits) to represent “INT8,” which we refer to as MXINT8 in QLLama. Fig. 4 illustrates an example of converting FP32 to MXINT8, involving three core stages: First, the sign bit of the original floating-point value is preserved to maintain positive/negative integrity. Second, extract the exponent bits of grouped elements, then generate a block-level shared scaling factor by adding the maximum exponent value with a preset offset (e.g., the maximum exponent 132 within a block plus the MXINT8 format offset of -1 results in 133). Finally, the mantissa is dynamically right-shifted based on the difference between each element’s exponent and the shared scaling factor, and the high-order valid bits after shifting are truncated. This scheme eliminates storage redundancy through exponent sharing and achieves hardware-efficient mapping via purely combinational logic (shift operations), circumventing the resource overhead of division operations in conventional quantization schemes.

The hardware of the QU realizes dynamic activation quantization through a two-stage task-level pipeline: shared factor calculation (Fig. 3 “Exp Scaling”) and dynamic shift scaling

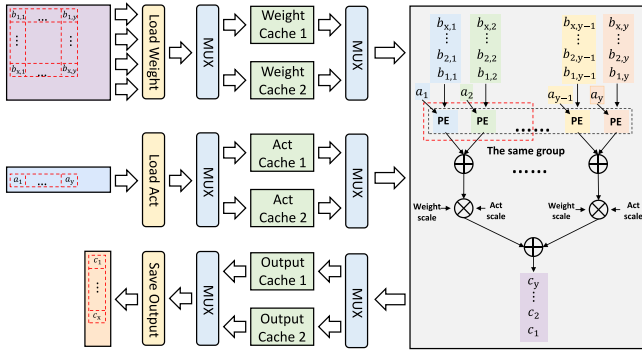


Fig. 5. Architecture of VMMU: integrated with weight load unit, input load unit, multiplexer, output save unit, and compute unit.

(Fig. 3 “Shift Quant”). First, the exponent and mantissa of floating-point data are separated, and a shared scaling factor is calculated based on block-wise maximum exponents; subsequently, according to the difference between each element’s exponent and the shared scaling factor, right shift operations are performed on mantissas to generate 8-bit fixed-point integers. A dual-mode processing scheme of low bit truncation and value wrapping is adopted for overflow data, finally outputting quantized data and an 8-bit unsigned shared factor. The shared factor calculation stage and the dynamic shift scaling stage form a producer-consumer model to achieve perfect overlap of the two-stage pipeline.

C. Vector-Matrix Multiplication Unit

For efficient computation of quantized weights, a dedicated VMMU is designed. Fig. 5 illustrates the implementation details of the VMMU, which consists of a weight load unit, an input load unit, a multiplexer, an output save unit, and a compute unit. Four optimizations are adopted in the design: block computation, computational optimization, ping-pong buffer optimization, and memory access optimization.

1) *Block Computation*: As shown in Fig. 5, the VMMU partitions the input dimension m and output dimension n into blocks with sizes $y = 1024$ and $x = 512$, respectively. During the computation phase, the activation vector sub-block $a_1 \cdots a_y$ and weight matrix sub-block $b_{11} \cdots b_{xy}$ undergo parallel dot product accumulation through a block multiplication accumulator to generate intermediate results $c_1 \cdots c_x$. Partial sum reduction across sub-block is achieved via the accumulator, requiring a total of $(m/y) \cdot (n/x)$ sub-block computations to finally output an n -dimensional result vector.

2) *Computational Optimization*: A total of 1024 processing elements (PEs) are instantiated, each containing one multiplier and one register to enable 1024-way parallel multiply accumulate operations. The data flow employs an input stay mechanism and a fifo structure: the activation vector a_i is temporarily stored within the i th PE, while the i th column of weight parameters b flows into the i th PE sequentially via the fifo. The results computed by the i th PE flow downward into an accumulation tree for further processing, where dequantization is implemented using bit shifting during the accumulation tree computation to effectively save resources.

TABLE I
HARDWARE PARAMETERS OF CPU, GPU, AND FPGA PLATFORMS

	CPU	GPU	FPGA
Platform	AMD EPYC 7543	NVIDIA RTX A6000	Xilinx Alveo U280
Frequency	2800MHz	1410MHz	200MHz
Computing Units	64 Cores	10752 Tensor Cores	9024 DSPs
Memory	256GB	48GB	8 & 32GB
Bandwidth	204.8GB/s	768GB/s	460 & 38GB/s

3) *Ping-Pong Buffer Optimization*: A double buffering mechanism with multiplexer circuits enables pipelined execution parallelism. The computing unit fetches current data blocks from the active buffer bank while prefetching operations concurrently load the subsequent weights and activations into the standby buffer bank, simultaneously writing back the computation results from the preceding cycle. The three stage pipeline (load-compute-writeback) achieves full operational overlap through dynamic buffer switching, thereby completely eliminating pipeline bubbles in data transmission.

4) *Memory Access Optimization*: Weight data is packaged to fully leverage the bus width, and the weight matrix is stored row-wise in 16 pseudo channels of HBM. 16-way AXI master interfaces are used for parallel reading, combined with pipelined prefetching to initiate read requests for subsequent blocks in advance, hiding HBM latency within computation cycles.

D. Scaled Dot-Product Attention Unit

The SPAU is responsible for computing the core operation of $\text{Softmax}(QK^T / \sqrt{d}) \cdot V$. To enhance efficiency, this unit optimizes the computation process into a three stage pipeline task: QK dot product computation (Fig. 3 “QK Dot”), softmax normalization (Fig. 3 “Softmax”), and weighted output (Fig. 3 “Attn Out”). It achieves efficient execution through block computation, parallel computing, and pipeline scheduling techniques.

E. Basic Operators Unit

The BOU integrates operations, such as RMSNorm, RoPE, and SiLU. For RMSNorm, the variance calculation stage, normalization coefficient generation stage, and weight scaling stage are implemented with task-level parallelism; RoPE pre-computes rotation factor tables ($\cos m\theta_i$, $\sin m\theta_i$) and achieves position encoding in real time through complex multiplication; SiLU and residual connections are pipelined.

IV. EVALUATION

A. Experiment Setup

We selected the Llama2-7B model as the experimental model and tested the average results of generating the first 32 tokens as experimental data. Perplexity comparisons between QLLama and the original model were conducted on the WikiText-2 dataset to ensure consistency in evaluation. Table I presents the hardware parameters of the GPU/CPU platform and QLLama. Our proposed QLLama is evaluated on the Xilinx Alveo U280 FPGA platform using Vitis HLS and Vivado. The

TABLE II
HARDWARE UTILIZATION OF ACCELERATOR ON ALVEO U280

	LUT	FF	BRAM	URAM	DSP
VMMU	128k (9.8%)	186k (7.1%)	512 (12.7%)	21 (2.2%)	674 (7.5%)
QU	8k (0.6%)	4k (0.1%)	0	0	0
SPAU	114k (8.7%)	122k (4.7%)	3 (0.1%)	0	781 (8.6%)
BOU	31k (2.4%)	15k (0.6%)	0	2 (0.2%)	56 (0.6%)
Buffer	67k (5.1%)	73k (2.8%)	556 (13.8%)	129 (13.4%)	4 (0.1%)
Total	348k (26.7%)	400k (15.3%)	1071 (26.6%)	152 (15.8%)	1515 (16.8%)

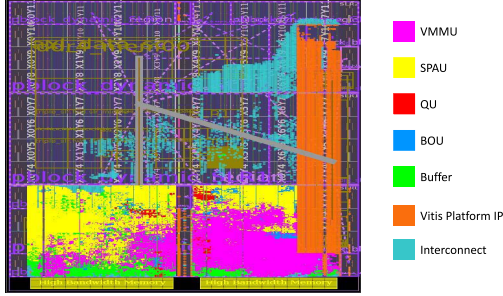


Fig. 6. Accelerator implementation layout on U280 FPGA.

TABLE III
LLAMA2-7B INFERENCE PERFORMANCE COMPARISON AT BATCH SIZE 1

	EPYC 7543	HLSTransform [7]	A6000	Ours
Freq. (GHz)	2.8	0.2	1.4	0.2
Lat. (ms)	160.34	\	19.98	52.79
Thput. (GOPS)	41.21	6.25	330.63	125.15
Power (W)	186.75	9.00	300.00	53.22
Eff. (GOPS/W)	0.2206	0.6949	1.1021	2.3517

resource consumption and layout routing results are shown in Table II and Fig. 6, and our kernel operates at 200 MHz. For comparison, we implemented the corresponding GPU version on NVIDIA RTX A6000 using PyTorch(v2.5.1) and CUDA(v12.4). Our CPU baseline is the original Llama2-7B model with FP32 precision running on the AMD EPYC 7543 server CPU. Furthermore, we also selected a domain-specific accelerator named HLSTransform [7] on the VU9P FPGA, which is designed to accelerate the inference tasks of the Llama2 model.

B. Experiments Results and Analysis

As shown in Table III, QLLama achieves a throughput of 125.15 GOPS, representing a 20 $\times$ improvement over HLSTransform. In terms of computational efficiency, QLLama outperforms the GPU, HLSTransform, and CPU by 2.13 $\times$, 3.38 $\times$, and 10.66 $\times$, respectively. These performance gains are achieved with negligible accuracy degradation, as evidenced by a perplexity increase of less than 0.2 (Table IV). The improvement comes primarily from two complementary optimizations. First, a hardware-friendly MXINT8 quantization scheme significantly reduces computational complexity and memory bandwidth. Second, custom-designed accelerator units are tailored to efficiently execute MXINT8 operations,

TABLE IV
PERPLEXITY COMPARISON OF LLAMA2-7B ON WIKETEXT-2

Module type	Perplexity $\downarrow$	Memory (GB) $\downarrow$	Difference
Original model (FP16)	9.393	12.55	—
Ours (MXINT8)	9.557	6.85	$\uparrow$ 1.74% / $\downarrow$ 45.4%

enabling fine-grained compute parallelism and optimized memory access.

V. CONCLUSION

To address the computational power and energy efficiency challenges in Llama2 large-model inference, this letter proposes and implements QLLama, an FPGA-based hardware-software co-design solution. The core innovations include: an innovative hardware-friendly MX data format quantization method that enables quantization/dequantization via shift operations only, significantly reducing hardware overhead; a mixed precision quantization strategy that optimizes computational efficiency while maintaining minimal accuracy loss (perplexity increase <0.2); and a customized QLLama accelerator architecture featuring a VMMU optimized for quantized weights and a dynamic QU. Experimental results demonstrate that QLLama achieves significant energy efficiency improvements on FPGA platforms (up to 10.66 $\times$ /2.13 $\times$ /3.38 $\times$ compared to CPU/GPU/HLSTransform) while preserving high model accuracy. QLLama provides an efficient solution for deploying large models at the edge.

REFERENCES

- [1] H. Touvron et al., “Llama 2: Open foundation and fine-tuned chat models,” 2023, *arXiv:2307.09288*.
- [2] Y. Qin, W. Lou, C. Wang, L. Gong, and X. Zhou, “Enhancing long sequence input processing in FPGA-based transformer accelerators through attention fusion,” in *Proc. GLVLSI*, 2024, pp. 599–603.
- [3] W. Lou, L. Gong, C. Wang, Z. Du, and X. Zhou, “OctCNN: A high throughput FPGA accelerator for CNNs using octave convolution algorithm,” *IEEE Trans. Comput.*, vol. 71, no. 8, pp. 1847–1859, Aug. 2022.
- [4] E. Frantar, S. Ashkboos, T. Hoeffer, and D. Alistarh, “GPTQ: Accurate post-training quantization for generative pre-trained transformers,” in *Proc. ICLR*, 2023, pp. 1–16.
- [5] C. Lee, J. Jin, T. Kim, H. Kim, and E. Park, “OWQ: Outlier-aware weight quantization for efficient fine-tuning and inference of large language models,” in *Proc. AAAI Conf. Artif. Intell.*, 2024, pp. 13355–13364.
- [6] B. D. Rouhani et al., “Microscaling data formats for deep learning,” 2023, *arXiv:2310.10537*.
- [7] A. He, D. Key, M. Bulling, A. Chang, S. Shapiro, and E. Lee, “HLSTransform: Energy-efficient Llama 2 inference on FPGAs via high level synthesis,” in *Proc. Workshop ICML*, 2024, pp. 1–9.
- [8] A. Vaswani et al., “Attention is all you need,” in *Proc. 31st Adv. Neural Inf. Process. Syst.*, 2017, pp. 1–11.
- [9] J. Ainslie, J. Lee-Thorp, M. d. Jong, Y. Zemlyanskiy, F. Lebrón, and S. Sanghai, “GQA: Training generalized multi-query transformer models from multi-head checkpoints,” 2023, *arXiv:2305.13245*.
- [10] S. Elfving, E. Uchibe, and K. Doya, “Sigmoid-weighted linear units for neural network function approximation in reinforcement learning,” *Neural Netw.*, vol. 107, pp. 3–11, Nov. 2018.
- [11] B. Zhang and R. Sennrich, “Root mean square layer normalization,” in *Proc. 33rd Adv. Neural Inf. Process. Syst.*, 2019, pp. 1–14.
- [12] K. He, X. Zhang, S. Ren, and J. Sun, “Deep residual learning for image recognition,” in *Proc. CVPR*, 2016, pp. 770–778.
- [13] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu, “RoFormer: Enhanced transformer with rotary position embedding,” *Neurocomputing*, vol. 568, Feb. 2024, Art. no. 127063.